

Quicksort

First Example

0	1	2	3	Array Index
23	12	25	5	Remarks
Pivot Left			Right	Increase Left until we find a number greater than Pivot or Left reaches extreme right. Decrease Right until we find number less than or equal to Pivot
		Left	Right	Since Left < Right Swap Left and Right
23	12	5	25	
		Right	Left	Since Left >= Right Swap Pivot and Right
(5	12)	(23)	(25)	
Pivot Left	Right			
Right	Left			Since Left >= Right Swap Pivot and Right
(5)	(12)	(23)	(25)	
	Pivot Left Right			Since Left >= Right Swap Pivot and Right
(5)	(12)	(23)	(25)	
			Pivot Left Right	Since Left >= Right Swap Pivot and Right
(5)	(12)	(23)	(25)	Sorted Data

Second Example

0	1	2	3	Array Index
27	23	8	8	Remarks
Pivot Left			Right	Increase Left until we find a number greater than Pivot or Left reaches extreme right. Decrease Right until we find number less than or equal to Pivot
			Left Right	Since Left >= Right Swap Pivot and Right
(8	23	8)	(27)	
Pivot Left		Right		
	Left	Right		Since Left < Right Swap Left and Right
8	8	23	(27)	
	Right	Left		Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	
Pivot Left Right				Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	
		Pivot Left Right		Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	Sorted Data

Case	Time Complexity
Worst-case performance	$O(n^2)$
Best-case performance	$O(n \log n)$
Average-case performance	$O(n \log n)$

Quicksort Program

```
#include <stdio.h>
void quicksort (int [], int, int);
int main()
{
    int array[25];
    int size, i;

    printf("Enter the number of elements: ");
    scanf("%d", &size);
    printf("Enter the elements to be sorted:\n");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &array[i]);
    }
    quicksort(array, 0, size - 1);
    printf("Ascending order list after applying quick sort:\n");
    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }
    printf("\n");

    return 0;
}
void quicksort(int array[], int low, int high)
{
    int pivot, left, right, temp;
    if (low < high)
    {
        pivot = low;
        left = low;
        right = high;
```

```

while (left < right)
{
/*Increase left until an element greater than pivot is found*/
while (array[left] <= array[pivot] && left <= high)
{
left++;
}

/*Decrease right until an element less than or equal to pivot is found*/
while (array[right] > array[pivot] && right >= low)
{
right--;
}

/*
if left<right, then swap array[left] and array[right]
*/
if (left < right)
{
temp = array[left];
array[left] = array[right];
array[right] = temp;
}
}
/*
if left==right or left>right then swap array[right] and array[pivot]
*/
temp = array[right];
array[right] = array[pivot];
array[pivot] = temp;
quicksort(array, low, right - 1); // Apply quicksort on the left side of pivot
quicksort(array, right + 1, high); // Apply quicksort on the right side of pivot
}
}

```

Quick Sort

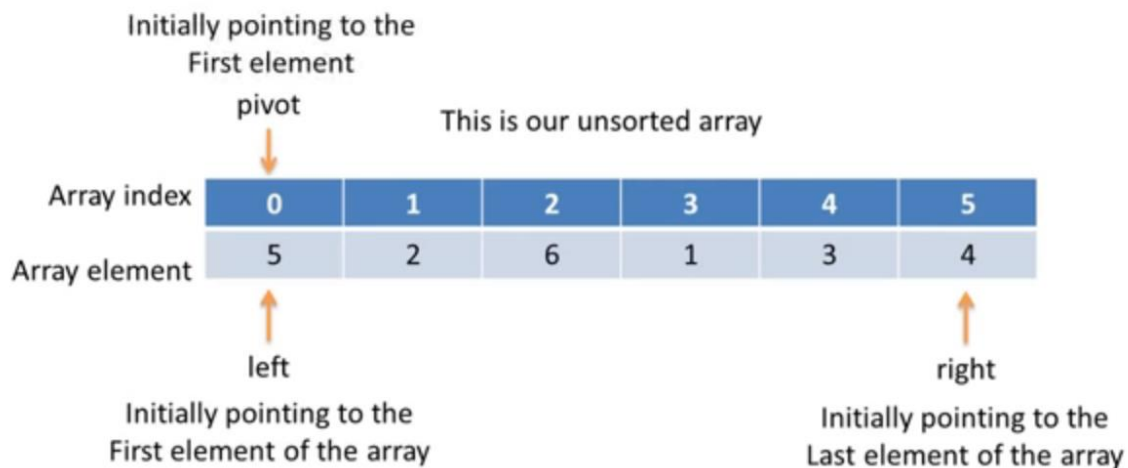
This sorting algorithm uses the idea of divide and conquer.

*It finds the element called **pivot** which divides the array into two halves in such a way that elements in the left half are smaller than pivot and elements in the right half are greater than pivot.*

Quick Sort

We follow three steps recursively:

- 1. Find pivot that divides the array into two halves.*
- 2. Quick sort the left half.*
- 3. Quick sort the right half.*

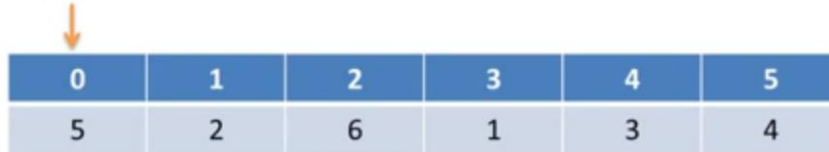


Remember this rule:

All elements to the **right** of pivot must be **greater** than pivot.

All elements to the **left** of pivot must be **smaller** than pivot.

pivot



0	1	2	3	4	5
5	2	6	1	3	4

left

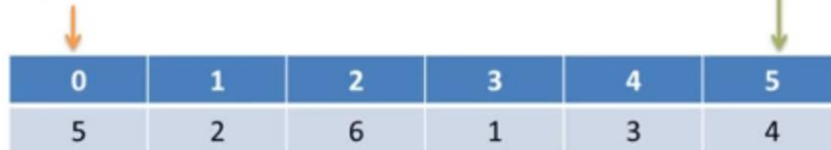
right

As the pivot
is pointing at
left

And move towards left

So we will start
from right

pivot



0	1	2	3	4	5
5	2	6	1	3	4

left

right

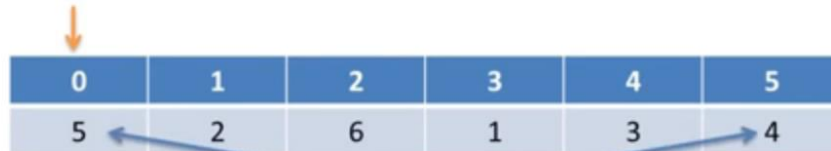
Is pivot < right?

NO

pivot = 5
right = 4

So we swap pivot and right

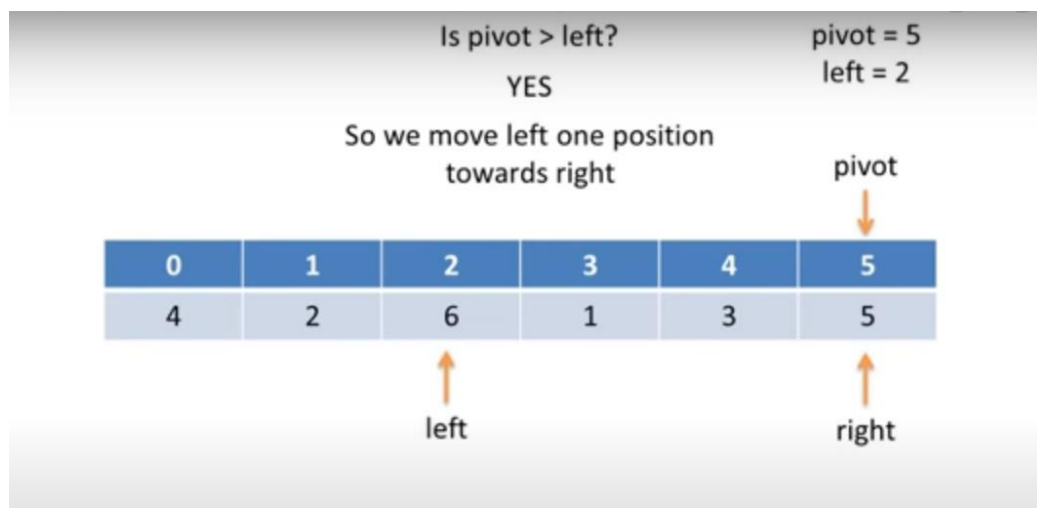
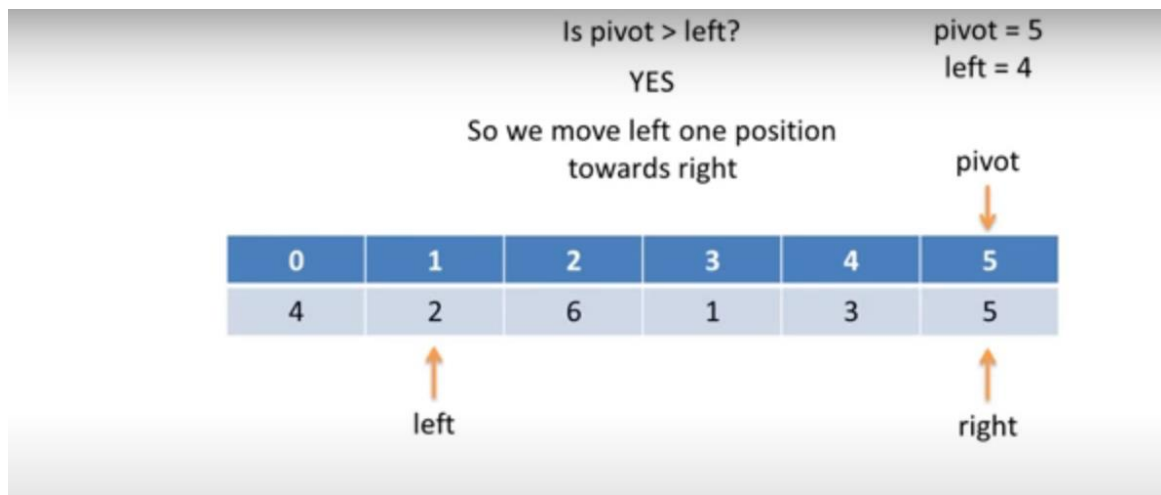
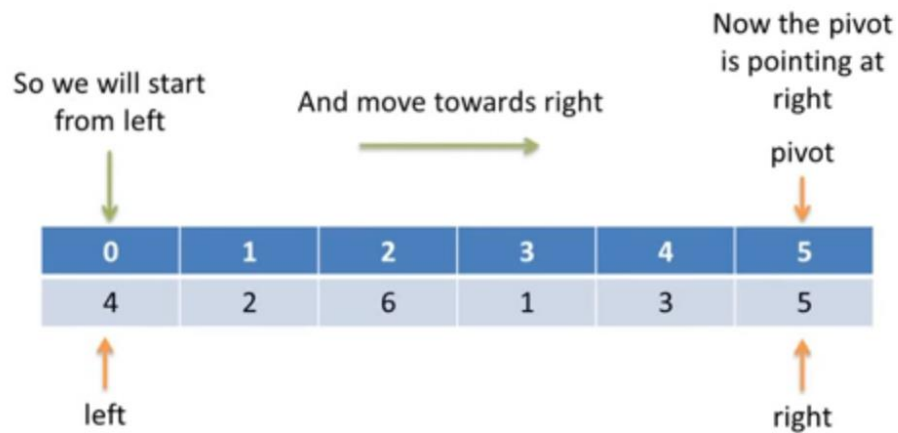
pivot



0	1	2	3	4	5
5	2	6	1	3	4

left

right



Is pivot > left?

NO

So we swap pivot and left
pivot

0	1	2	3	4	5
4	2	5	1	3	6

And move the pivot to left

Is pivot < right?

YES

So we move right one position towards left
pivot

pivot = 5

right = 6

0	1	2	3	4	5
4	2	5	1	3	6

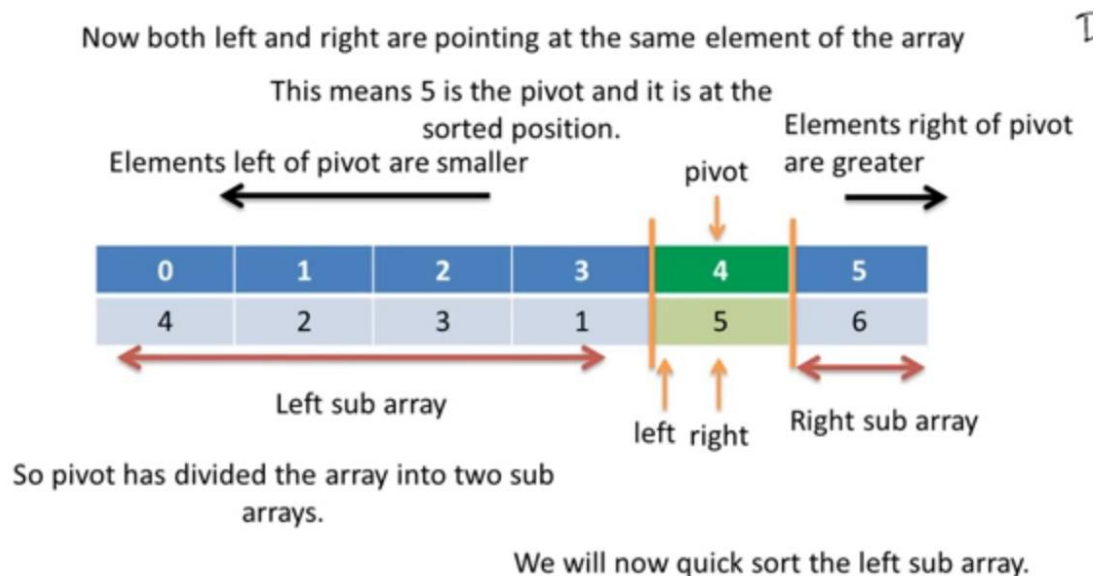
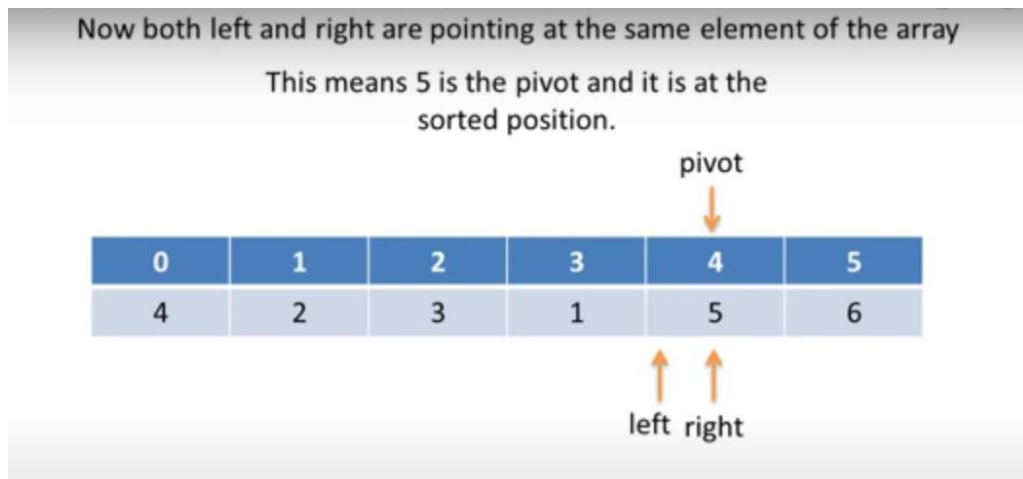
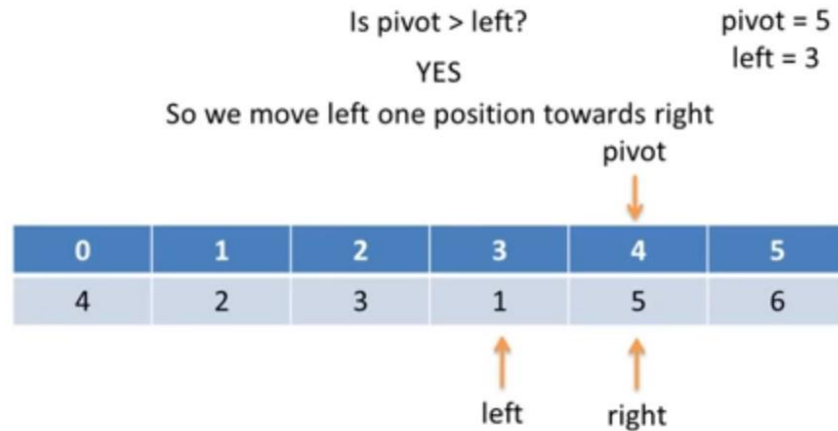
Is pivot < right?

NO

So we swap pivot and right
pivot

0	1	2	3	4	5
4	2	3	1	5	6

And move the pivot to right



Algorithm

```
/*a[0:n-1] is an array of n elements.  
beg = first index of array  
end = last index of array  
*/  
QuickSort(a, beg, end)  
Begin  
    if beg < end then  
        Call PartitionArray(a, beg, end, pivotLoc); //this will find pivot location  
        Call QuickSort(a, beg, pivotLoc - 1); //Quick sort left sub array  
        Call QuickSort(a, pivotLoc + 1, end); //Quick sort right sub array  
    endif  
End
```

//Quicksort in C

```
#include <stdio.h>  
  
#define SIZE 10 //array size  
  
void partitionArray(int *a, int beg, int end, int *pivotLoc);  
void quickSort(int *a, int beg, int end);  
  
int main()  
{  
    int a[] = {5,2,6,1,3,4,2,5,7}; //unsorted array  
    int n=sizeof(a)/sizeof(a[0]);  
    int i;  
  
    quickSort(a, 0, n-1); //beg = 0 start index of array end = 5 last index of array  
  
    //printing sorted element of array  
    for(i = 0; i < n; i++){  
        printf("%d\t", a[i]);  
    }  
    return 0;  
}  
  
}//main() ends here
```

```

void quickSort(int *a, int beg, int end)
{
    int pivotLoc;
    if(beg < end)
    {
        partitionArray(a, beg, end, &pivotLoc); //this will find the pivot location and
        partition the array

        quickSort(a, beg, pivotLoc - 1); //quick sort the left sub array
        quickSort(a, pivotLoc + 1, end); //quick sort the right sub array
    }
} //quickSort() ends here

void partitionArray(int *a, int beg, int end, int *pivotLoc)
{
    int left = beg;    //initially left point to the first element of the array
    int right = end;   //initially right point to the last element of the array
    *pivotLoc = left;  //initially pivot point to first element of the array
    int tmp;           //used for swapping values
    while(1)
    {
        //pivot pointing at left
        while(a[*pivotLoc] <= a[right] && *pivotLoc != right)
        { //pivot element <= right element
            right--; //move right one position towards left
        }
        if(*pivotLoc == right)
        { //both left and right pointing at same element of the array
            break;
        }
        else if(a[*pivotLoc] > a[right])
        {
            //pivot element greater than right element. swap pivot and right element.
            tmp = a[right];

```

```

    a[right] = a[*pivotLoc];
    a[*pivotLoc] = tmp;
    *pivotLoc = right; //pivot is now pointing to right
}
//pivot pointing to right
while(a[*pivotLoc] >= a[left] && *pivotLoc != left)
{ //pivot element >= left element
    left++; //move left one position towards right
}
if(*pivotLoc == left)
{ //both left and right pointing at the same element of the array
    break;
}
else if(a[*pivotLoc] < a[left])
{
    //pivot element smaller than left element. swap pivot and left element.
    tmp = a[left];
    a[left] = a[*pivotLoc];
    a[*pivotLoc] = tmp;
    *pivotLoc = left; //pivot is now pointing to left
}
} //end of while(1)
} //partitionArray() ends here

```